Intelligent Systems

Student Evaluation of Teaching classification

Intelligent Systems

Student Evaluation of Teaching classification

1. Context & Evolution

Background

Analysis of open-ended teaching-evaluation comments.

Technological Progression

Tracked from traditional lexicons and conventional classifiers to modern transformers and large language models (LLMs).

2. The Challenge

Current Bottleneck

Despite advanced models, real-world practical adoption remains limited.

Key Deficiencies

- Implementation is scarce
- Usage in institutional workflows is low
- Actionability of results remains limited

3. Project Objective

Midterm Scope

Evaluated an initial Machine Learning (ML) classification model for teaching feedback.

Project Deliverable

Integrated the evaluated model into a functional Minimum Viable Product (MVP) to bridge the actionability gap.

Intelligent Systems

Student Evaluation of Teaching classification

ChatGPT / Claude

- Diseño de criterios de etiquetado
- Identificación de ambigüedades
- Análisis cualitativo de errores

NotebookLM

- Sintetizar documentación
- Literatura metodológica suministrada

Hugging Face

- Explorar modelos de embeddings
- Comparar representaciones semánticas con TF-IDF

scikit-learn

- Entrenamiento y validación cruzada
- Ajuste de hiperparámetros
- Evaluación reproducible

Rol de los LLM y Consideraciones Metodológicas

Los LLM deben funcionar como **asistentes metodológicos**. Las etiquetas generadas automáticamente necesitan **validación humana**, y los comentarios estudiantiles deben **anonimizarse** antes de enviarlos a servicios externos.

Intelligent Systems

Student Evaluation of Teaching classification

1. Carga y exploración del CSV
Verificar columnas, etiquetas, faltantes, duplicados y calidad.
✦ LLM: ChatGPT/Claude para explorar esquema y calidad.

2. Definición de la tarea y preprocesamiento
Normalizar texto, tratar faltantes, emojis e intensificadores.
✦ LLM: Guías de etiquetado y revisión de casos ambiguos.

3. División de datos
Separar train (80%) y test (20%) estratificados. Evitar fuga de información y agrupar por autor.

4. Representación vectorial del texto
Comparar TF-IDF vs. embeddings semánticos solo con train.
✦ LLM: Comparar alternativas y analizar errores lingüísticos.

5. Entrenamiento inicial y línea base
Evaluar DummyClassifier de referencia y comparar Logistic Regression, Linear SVM, Naive Bayes y Random Forest.

6. Ajuste de hiperparámetros
Pipeline + GridSearchCV/RandomizedSearchCV con CV estratificada. Seleccionar por macro-F1.
✦ LLM: Sugerir rangos e interpretar resultados.

7. Evaluación final y comparación
Evaluar sobre test: accuracy, precision, recall, macro-F1 y matriz de confusión.
✦ LLM: Resumir hallazgos y categorizar errores.

Submission: Use the template from https://revistas.javeriana.edu.co/index.php/iyu/about/submissions and the article submission guidelines for the journal Ingeniería y Universidad

Intelligent Systems

Student Evaluation of Teaching classification

1. Seleccionar el mejor modelo

Elegir el modelo según los resultados de validación y evaluación en test. Registrar sus métricas, parámetros y versión del conjunto de datos.

2. Guardar el modelo con Joblib

Guardar en .joblib serializando un Pipeline completo (vectorizador + clasificador) para conservar el preprocesamiento.

✦ Herramienta: joblib · Registrar versión de Python y dependencias.

3. Validar la carga y las predicciones

Cargar el archivo guardado en un entorno limpio y verificar coherencia en predicciones. Probar textos vacíos, errores y comentarios ambiguos.

4. Crear un servicio de predicción

Desarrollar una API con FastAPI o Flask que cargue el modelo y reciba comentarios. Retornar la clase predicha y probabilidades por clase.

5. Integrar con la aplicación

Crear una interfaz con Streamlit, app web o sistema institucional. Permitir ingresar comentarios individuales o cargar archivos CSV.

6. Probar, desplegar y monitorear

Verificar tiempos de inferencia, seguridad y calidad. Desplegar en servidor o plataforma cloud y monitorizar cambios en los datos.

LLM: Apoyar la documentación, generar casos de prueba y resumir errores; no exponer comentarios identificables.

Intelligent Systems

Student Evaluation of Teaching classification

1. Pipeline unificado

Guardar el vectorizador y el clasificador juntos en un único Pipeline, incluyendo cualquier transformación de texto necesaria.

2. Eficiencia de entrenamiento

No volver a entrenar el modelo cada vez que se inicia la aplicación.

3. Seguridad en deserialización

Cargar únicamente archivos Joblib de confianza: la deserialización puede ejecutar código arbitrario.

4. Compatibilidad de versiones

Mantener las mismas versiones compatibles de scikit-learn y sus dependencias entre entrenamiento y producción.

5. Salida estructurada

Si se predicen nueve aspectos, definir un formato de salida estructurado con una predicción por aspecto.

6. Control y reversión de versiones

Registrar las versiones del modelo y disponer de un procedimiento para actualizarlo o revertirlo.

Herramientas sugeridas: joblib para persistencia, scikit-learn para inferencia, Streamlit para una app rápida y FastAPI si se necesita separar el frontend del backend.

Intelligent Systems

Student Evaluation of Teaching classification

Intelligent Systems

Student Evaluation of Teaching classification